04장 파이썬의 입출력

지금까지 공부한 내용을 바탕으로 함수, 입출력, 파일 처리 방법에 대해 알아보자. 입출력은 프로그래밍 설계와 밀접한 관련이 있다. 프로그래머는 프로그램을 만들기 전에 어떻게 동작하게 할 것인지 설계부터 하는데, 이때 가장 중요한 부분이 바로 입출력이다. 특정 프로그램에서만 사용하는 함수를 만들 것인지, 여러 프로그램이 공통으로 사용하는 함수를 만들 것인지, 더 나아가 오픈 API로 공개하여 외부 프로그램도 사용할 수 있게 만들 것인지가 모두 입출력과 관련 있다.

목차

- 04-1 함수
- 04-2 사용자 입출력
- 04-3 파일 읽고 쓰기
- 04-4 프로그램의 입출력
- 04장 되새김 문제

04-1 함수

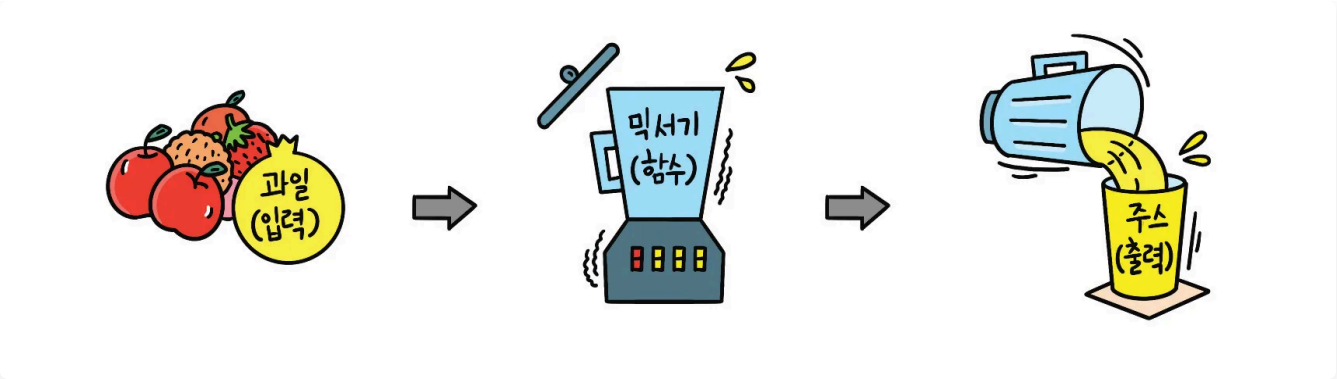
파이썬에서 함수를 이해하는 것은 매우 중요하다. 먼저 함수란 무엇인지 알아보자.

- 함수란 무엇인가?
- 함수를 사용하는 이유는 무엇일까?
- 파이썬 함수의 구조
- 매개변수와 인수
- 입력값과 반환값에 따른 함수의 형태
    - 일반적인 함수
    - 입력값이 없는 함수
    - 반환값이 없는 함수
    - 입력값도, 반환값도 없는 함수
- 매개변수를 지정하여 호출하기
- 입력값이 몇 개가 될지 모를 때는 어떻게 해야 할까?
    - 여러 개의 입력값을 받는 함수 만들기
    - 키워드 매개변수, kwargs
- 함수의 반환값은 언제나 하나이다
- 매개변수에 초깃값 미리 설정하기
- 함수 안에서 선언한 변수의 효력 범위
- 함수 안에서 함수 밖의 변수를 변경하는 방법
    - 1. return 사용하기
    - 2. global 명령어 사용하기
- lambda 예약어
- 함수의 독스트링(Docstring)

함수란 무엇인가?

함수를 설명하기 전에 믹서를 생각해 보자. 우리는 믹서에 과일을 넣는다. 그리고 믹서를 사용해서 과일을 갈아 과일 주스를 만든다. 우리가 믹서에 넣는 과일은 '입력', 과일 주스는 '출력(결괏값)'이 된다.

그렇다면 믹서는 무엇일까?



> *믹서는 과일을 입력받아 주스를 출력하는 함수와 같다.*

우리가 배우려는 함수가 바로 믹서와 비슷하다. 입력값을 가지고 어떤 일을 수행한 후 그 결과물을 내어 놓는 것이 바로 함수가 하는 일이다. 예를 들어 $y = 2x + 3$도 함수이다. 입력값(x)에 따라 출력값(y)이 변하는 것, 이것이 바로 함수이다.

이제 우리는 함수에 대해 좀 더 생각해 봐야 한다. 프로그래밍에서 함수는 정말 중요하기 때문이다. 파이썬 함수의 세계로 깊이 들어가 보자.

함수를 사용하는 이유는 무엇일까?

프로그래밍을 하다 보면 똑같은 내용을 반복해서 작성하는 자신을 발견할 때가 종종 있다. 이때가 바로 함수가 필요한 때이다. 반복되는 부분이 있을 경우, '반복적으로 사용되는 가치 있는 부분'을 한 뭉치로 묶어 '어떤 입력값을 주었을 때 어떤 결괏값을 반환해 준다'라는 식의 함수로 작성하는 것이다.

함수를 사용하는 또 다른 이유는 프로그램을 기능 단위의 함수로 분리해 놓으면 프로그램 흐름을 일목요연하게 볼 수 있기 때문이다. 마치 공장에서 원재료가 여러 공정을 거쳐 하나의 완제품이 되는 것처럼 프로그램에서도 입력한 값이 여러 함수를 거치면서 원하는 결괏값을 내는 것을 볼 수 있다. 이렇게 되면 프로그램 흐름도 잘 파악할 수 있고 오류가 어디에서 나는지도 쉽게 알아차릴 수 있다.

파이썬 함수의 구조

파이썬 함수의 구조는 다음과 같다.

```
def 함수_이름(매개변수):
    수행할_문장1
    수행할_문장2
    ...
```

def는 함수를 만들 때 사용하는 예약어이며, 함수 이름은 함수를 만드는 사람이 임의로 만들 수 있다. 함수 이름 뒤 괄호 안의 매개변수는 이 함수에 입력으로 전달되는 값을 받는 변수이다. 이렇게 함수를 정의한 후 if, while, for 문 등과 마찬가지로 함수에서 수행할 문장을 입력한다.

간단하지만 많은 것을 설명해 주는 다음 예를 살펴보자.

```
def add(a, b):
    return a + b
```

위 함수는 다음과 같이 풀이된다.

```
이 함수의 이름은 add이고 입력으로 2개의 값을 받으며 반환값(출력값)은 2개의 입력값을 더한 값이다.
```

여기에서 return은 함수의 결괏값(반환값)을 반환하는 명령어이다. 먼저 다음과 같이 add 함수를 만들자.

```
>>> def add(a, b):
...     return a+b
...
>>>
```

이제 직접 add 함수를 사용해 보자.

```
>>> a = 3
>>> b = 4
>>> c = add(a, b)  # add(3, 4)의 반환값을 c에 대입
>>> print(c)
7
```

변수 a에 3, b에 4를 대입한 후 앞에서 만든 add 함수에 a와 b를 입력값으로 넣어 준다. 그리고 변수 c에 add 함수의 반환값을 대입하면 print(c)로 c의 값을 확인할 수 있다.

매개변수와 인수

매개변수(parameter)와 인수(arguments)는 혼용해서 사용하는 용어이므로 잘 기억해 두자. 매개변수는 함수에 입력으로 전달된 값을 받는 변수, 인수는 함수를 호출할 때 전달하는 입력값을 의미한다.

```
def add(a, b):  # a, b는 매개변수
    return a+b

print(add(3, 4))  # 3, 4는 인수
```

점프 투 파이썬

같은 의미를 가진 여러 가지 용어에 주의하자

프로그래밍을 공부할 때 어려운 부분 중 하나가 용어의 혼용이다. 공부하면서 원서를 보기도 하고 번역본을 보기도 하면서 의미는 같지만 표현이 다른 용어를 자주 만나게 된다. 예를 들어 입력값을 함수의 인수, 파라미터, 매개변수 등으로 말하기도 하고, 함수의 반환값을 결괏값, 출력값, 리턴값, 돌려 주는 값 등으로 말하기도 한다. 이렇듯 많은 용어가 여러 가지 다른 말로 표현되지만 의미는 동일한 경우가 많다. 이런 용어를 기억해 두면 나중에 혼란을 줄일 수 있다.

> *이 책에서는 함수를 사용할 때 입력값, 반환값이라는 용어로 통일하여 사용한다.*

입력값과 반환값에 따른 함수의 형태

함수는 들어온 입력값을 받은 후 어떤 처리를 하여 적절한 값을 반환해 준다.



함수의 형태는 입력값과 반환값의 존재 유무에 따라 4가지 유형으로 나뉜다. 자세히 알아보자.

일반적인 함수

입력값이 있고 반환값이 있는 함수가 일반적인 함수이다. 앞으로 여러분이 프로그래밍을 할 때 만들 함수는 대부분 다음과 비슷한 형태일 것이다.

```
def 함수_이름(매개변수):
    수행할_문장
    ...
    return 반환값
```

다음은 일반적인 함수의 전형적인 예이다.

```
def add(a, b):
    result = a + b
    return result
```

add 함수는 2개의 입력값을 받아 서로 더한 결괏값을 반환한다.

이 함수를 사용하는 방법은 다음과 같다. 입력값으로 3과 4를 주고 반환값을 받아 보자.

```
>>> a = add(3, 4)
>>> print(a)
7
```

이처럼 입력값과 반환값이 있는 함수의 사용법을 정리하면 다음과 같다.

```
반환값을_받을_변수 = 함수_이름(입력_인수1, 입력_인수2, ...)
```

입력값이 없는 함수

입력값이 없는 함수가 존재할까? 당연히 존재한다. 다음과 같이 작성해 보자.

```
>>> def say():
...     return 'Hi'
```

say라는 이름의 함수를 만들었다. 그런데 매개변수 부분을 나타내는 함수 이름 뒤의 괄호 안이 비어 있다. 이 함수는 어떻게 사용하는 것일까?

다음을 직접 입력해 보자.

```
>>> a = say()
>>> print(a)
Hi
```

위 함수를 쓰기 위해서는 say()처럼 괄호 안에 아무런 값도 넣지 않아야 한다. 이 함수는 입력값은 없지만, 반환값으로 "Hi"라는 문자열을 반환한다. 즉, `a = say()` 처럼 작성하면 a에 "Hi"라는 문자열이 대입되는 것이다.

이처럼 입력값이 없고 반환값만 있는 함수는 다음과 같이 사용한다.

```
반환값을_받을_변수 = 함수_이름()
```

반환값이 없는 함수

반환값이 없는 함수 역시 존재한다. 다음 예를 살펴보자.

```
>>> def add(a, b):
...     print(f"{a}, {b}의 합은 {a+b}입니다.")
```

반환값이 없는 함수는 호출해도 반환되는 값이 없기 때문에 다음과 같이 사용한다.

```
>>> add(3, 4)
3, 4의 합은 7입니다.
```

즉, 반환값이 없는 함수는 다음과 같이 사용한다.

```
함수_이름(입력_인수1, 입력_인수2, ...)
```

여기서 '3, 4의 합은 7입니다.'라는 문장을 출력했는데 왜 반환값이 없다는 것인지 의아할 수도 있다. 초보자들이 혼란스러워하는 부분인데, print 문은 함수의 구성 요소 중 하나인 '수행할_문장'에 해당하는 부분일 뿐이다. 반환값은 당연히 없다. 반환값은 오직 return 명령어로만 반환받을 수 있다.

이를 확인해 보자. 반환받을 값을 a 변수에 대입하고 a 값을 출력해 보면 반환값이 있는지 없는지 알 수 있다.

```
>>> a = add(3, 4)
3, 4의 합은 7입니다.
>>> print(a)
None
```

a 값으로 None이 출력되었다. None은 값이 없음을 나타내는 객체이다. 참과 거짓을 따질 때는 False처럼 동작한다. add 함수처럼 반환값이 없을 때 `a = add(3, 4)`처럼 쓰면 함수 add는 반환값으로 a 변수에 None을 반환한다. None을 반환한다는 것은 반환값이 없다는 것이다.

입력값도, 반환값도 없는 함수

입력값도, 반환값도 없는 함수 역시 존재한다. 다음 예를 살펴보자.

```
>>> def say():
...     print('Hi')
```

입력 인수를 받는 매개변수도 없고 return 문도 없으니 입력값도, 반환값도 없는 함수이다.

이 함수를 사용하는 방법은 단 1가지이다.

```
>>> say()
Hi
```

즉, 입력값도, 반환값도 없는 함수는 다음과 같이 사용한다.

```
함수_이름()
```

매개변수를 지정하여 호출하기

함수를 호출할 때 매개변수를 지정할 수도 있다. 다음 예를 살펴보자.

```
>>> def sub(a, b):
...     return a - b
```

2개의 숫자를 입력받은 후 첫 번째 수에서 두 번째 수를 뺄셈하여 반환하는 sub 함수이다. 이 함수는 다음과 같이 매개변수를 지정하여 사용할 수 있다.

```
>>> result = sub(a=7, b=3)  # a에 7, b에 3을 전달
>>> print(result)
4
```

매개변수를 지정하면 다음과 같이 순서에 상관없이 사용할 수 있다는 장점이 있다.

```
>>> result = sub(b=5, a=3)  # b에 5, a에 3을 전달
>>> print(result)
-2
```

입력값이 몇 개가 될지 모를 때는 어떻게 해야 할까?

입력값이 여러 개일 때 그 입력값을 모두 더해 주는 함수를 생각해 보자. 하지만 몇 개가 입력될지 모를 때는 어떻게 해야 할까? 파이썬은 이런 문제를 해결하기 위해 다음과 같은 방법을 제공한다.

```
def 함수_이름(*매개변수):
    수행할_문장
    ...
```

일반적으로 볼 수 있는 함수 형태에서 괄호 안의 매개변수 부분이 `*매개변수` 로 바뀌었다.

여러 개의 입력값을 받는 함수 만들기

다음 예를 통해 여러 개의 입력값을 모두 더하는 함수를 직접 만들어 보자. 예를 들어 `add_many(1, 2)` 이면 3, `add_many(1, 2, 3)` 이면 6, `add_many(1, 2, 3, 4, 5, 6, 7, 8, 9, 10)` 이면 55를 반환하는 함수를 만들어 보자.

```
>>> def add_many(*args):
...     result = 0
...     for i in args:
...         result = result + i   # *args에 입력받은 모든 값을 더한다.
...     return result
```

위에서 만든 add_many 함수는 입력값이 몇 개이든 상관없다. `*args` 처럼 매개변수 이름 앞에 `*` 을 붙이면 입력값을 전부 모아 튜플로 만들어 주기 때문이다. 만약 `add_many(1, 2, 3)` 처럼 이 함수를 쓰면 args는 (1, 2, 3)이 되고 `add_many(1, 2, 3, 4, 5, 6, 7, 8, 9, 10)` 처럼 쓰면 args는 (1, 2, 3, 4, 5, 6, 7, 8, 9, 10)이 된다. 여기에서 `*args` 는 임의로 정한 변수 이름이다. `*pey` , `*python` 처럼 아무 이름이나 써도 된다.

> *args는 인수를 뜻하는 영어 단어 arguments의 약자이며 관례적으로 자주 사용한다.*

작성한 add_many 함수를 다음과 같이 사용해 보자.

```
>>> result = add_many(1,2,3)
>>> print(result)
6
>>> result = add_many(1,2,3,4,5,6,7,8,9,10)
>>> print(result)
55
```

`add_many(1, 2, 3)` 으로 함수를 호출하면 6을 반환하고 `add_many(1, 2, 3, 4, 5, 6, 7, 8, 9, 10)` 으로 함수를 호출하면 55를 반환하는 것을 확인할 수 있다.

여러 개의 입력을 처리할 때 `def add_many(*args)` 처럼 함수의 매개변수로 `*args` 하나만 사용할 수 있는 것은 아니다. 다음 예를 살펴보자.

```
>>> def add_mul(choice, *args):
...     if choice == "add":   # 매개변수 choice에 "add"를 입력받았을 때
...         result = 0
...         for i in args:
...             result = result + i
...     elif choice == "mul":   # 매개변수 choice에 "mul"을 입력받았을 때
...         result = 1
...         for i in args:
...             result = result * i
...     return result
```

> *인터프리터에서 작성하기에 소스 코드가 너무 많다면 에디터를 사용해도 된다.*

add_mul 함수는 여러 개의 입력값을 의미하는 `*args` 매개변수 앞에 choice 매개변수가 추가되어 있다.

이 함수는 다음과 같이 사용할 수 있다.

```
>>> result = add_mul('add', 1,2,3,4,5)
>>> print(result)
15
>>> result = add_mul('mul', 1,2,3,4,5)
>>> print(result)
120
```

매개변수 choice에 'add'가 입력된 경우 `*args` 에 입력되는 모든 값을 더해서 15를 반환하고 'mul'이 입력된 경우 `*args` 에 입력되는 모든 값을 곱해 120을 반환한다.

키워드 매개변수, kwargs

이번에는 키워드 매개변수에 대해 알아보자. 키워드 매개변수는 함수 호출 시 `키워드=값` 형태로 전달하는 매개변수를 받을 때 사용한다. 키워드 매개변수를 사용할 때는 매개변수 앞에 별 2개(`**`)를 붙인다.

먼저 간단한 예제로 시작해 보자.

```
>>> def print_kwargs(**kwargs):
...     print(kwargs)
```

print_kwargs는 입력받은 키워드 매개변수들을 딕셔너리 형태로 출력하는 함수이다. 이제 이 함수를 다음과 같이 사용해 보자.

```
>>> print_kwargs(a=1)
{'a': 1}
>>> print_kwargs(name='foo', age=3)
{'name': 'foo', 'age': 3}
>>> print_kwargs(name='홍길동', age=25, city='서울', job='개발자')
{'name': '홍길동', 'age': 25, 'city': '서울', 'job': '개발자'}
```

함수의 입력값으로 `a=1` 이 사용되면 kwargs는 `{'a': 1}` 이라는 딕셔너리가 되고, 입력값으로 `name='foo', age=3` 이 사용되면 kwargs는 `{'name': 'foo', 'age': 3}` 라는 딕셔너리가 된다. 즉, `**kwargs` 처럼 매개변수 이름 앞에 `**` 을 붙이면 매개변수 kwargs는 딕셔너리가 되고 모든 `키워드=값` 형태의 입력값이 그 딕셔너리에 저장된다.

키워드 매개변수의 실용적인 예를 살펴보자.

```
>>> def create_profile(**info):
...     print("=== 프로필 정보 ===")
...     for key, value in info.items():
...         print(f"{key}: {value}")
...
>>> create_profile(이름='김철수', 나이=30, 직업='프로그래머', 취미='독서')
=== 프로필 정보 ===
이름: 김철수
나이: 30
직업: 프로그래머
취미: 독서
```

이처럼 키워드 매개변수를 사용하면 함수 호출 시 어떤 키워드든 자유롭게 전달할 수 있어 매우 유연한 함수를 만들 수 있다.

일반 매개변수, 가변 매개변수(`*args`), 키워드 매개변수(`**kwargs`)를 모두 함께 사용할 수도 있다. 이때 순서는 반드시 다음과 같아야 한다.

```
>>> def mixed_function(name, *args, **kwargs):
...     print(f"이름: {name}")
...     print(f"추가 인수들: {args}")
...     print(f"키워드 인수들: {kwargs}")
...
>>> mixed_function('홍길동', 1, 2, 3, age=25, city='서울')
이름: 홍길동
추가 인수들: (1, 2, 3)
키워드 인수들: {'age': 25, 'city': '서울'}
```

> *kwargs는 'keyword arguments'의 약자이며 args와 마찬가지로 관례적으로 사용한다.*

함수의 반환값은 언제나 하나이다

먼저 다음 함수를 만들어 보자.

```
>>> def add_and_mul(a,b):
...     return a+b, a*b
```

add_and_mul은 2개의 입력 인수를 받아 더한 값과 곱한 값을 반환하는 함수이다.

이 함수를 다음과 같이 호출하면 어떻게 될까?

```
>>> result = add_and_mul(3,4)
```

반환값은 `a+b` 와 `a*b` 인데, 반환값을 받아들이는 변수는 result 하나만 쓰였으므로 오류가 발생하지 않을까? 당연한 의문이다. 하지만 오류는 발생하지 않는다. 그 이유는 함수의 반환값은 2개가 아니라 언제나 1개라는 데 있다. add_and_mul 함수의 반환값 `a+b` 와 `a*b` 는 튜플값 하나인 `(a+b, a*b)` 로 반환된다.

따라서 result 변수는 다음과 같은 값을 가지게 된다.

```
result = (7, 12)
```

즉, 결괏값으로 (7, 12)라는 튜플 값을 가지게 되는 것이다.

만약 이 하나의 튜플 값을 2개의 값으로 분리하여 받고 싶다면 함수를 다음과 같이 호출하면 된다.

```
>>> result1, result2 = add_and_mul(3, 4)
```

이렇게 호출하면 `result1, result2 = (7, 12)` 가 되어 result1은 7, result2는 12가 된다.

또 다음과 같은 의문이 생길 수도 있다.

```
>>> def add_and_mul(a,b):
...     return a+b
...     return a*b
```

위와 같이 return 문을 2번 사용하면 2개의 반환값을 돌려 주지 않을까? 하지만 기대하는 결과는 나오지 않는다.

그 이유는 add_and_mul 함수를 호출해 보면 알 수 있다.

```
>>> result = add_and_mul(2, 3)
>>> print(result)
5
```

`add_and_mul(2, 3)` 의 반환값은 5 하나뿐이다. 두 번째 return 문인 `return a * b` 는 실행되지 않았다는 뜻이다. 함수는 return 문을 만나는 순간, 반환값을 반환한 다음 함수를 빠져나간다.

따라서 이 함수는 다음과 완전히 동일하다.

```
>>> def add_and_mul(a,b):
...     return a+b
```

점프 투 파이썬

return의 또 다른 쓰임새

특별한 상황일 때 함수를 빠져나가고 싶다면 return을 단독으로 써서 함수를 즉시 빠져나갈 수 있다. 다음 예를 살펴보자.

```
>>> def say_nick(nick):
...     if nick == "바보":
...         return
...     print(f"나의 별명은 {nick}입니다.")
```

위는 매개변수 nick으로 별명을 입력받아 출력하는 함수이다. 이 함수 역시 반환값은 없다. 이때 문자열을 출력한다는 것과 반환값이 있다는 것은 전혀 다른 말이므로 혼동하지 말자. 함수의 반환값은 오로지 return 문에 의해서만 생성된다.

만약 입력값으로 '바보'라는 값이 들어오면 문자열을 출력하지 않고 함수를 즉시 빠져나간다.

```
>>> say_nick('야호')
나의 별명은 야호입니다.
>>> say_nick('바보')
>>>
```

이처럼 반환값이 없는 함수에서 return으로 함수를 빠져나가는 방법은 실제 프로그래밍에서 자주 사용한다.

매개변수에 초깃값 미리 설정하기

이번에는 조금 다른 형태로 함수의 인수를 전달하는 방법에 대해서 알아보자. 다음은 매개변수에 초깃값을 미리 설정해 주는 경우이다.

> *>>> (프롬프트) 표시가 없으니 IDLE 에디터에서 작성하자.*

```
# default1.py
def say_myself(name, age, man=True):
    print(f"나의 이름은 {name}입니다.")
    print(f"나이는 {age}살입니다.")
    if man:
        print("남자입니다.")
    else:
        print("여자입니다.")
```

위 함수를 보면 매개변수가 `name, age, man=True`이다. 그런데 낯선 것이 나왔다. `man=True`처럼 매개변수에 미리 값을 넣어 준 것이다. 이것이 바로 함수의 매개변수에 초깃값을 설정하는 방법이다.

say_myself 함수는 다음처럼 2가지 방법으로 사용할 수 있다.

```
say_myself("박응용", 27)
```

```
say_myself("박응용", 27, True)
```

입력값으로 ("박응용", 27)처럼 2개를 주면 name에는 "박응용", age에는 27이 대입된다. 그리고 man이라는 변수에는 입력값을 주지 않았지만, man은 초깃값 True를 갖게 된다.

따라서 위 예에서 say_myself 함수를 사용한 2가지 방법은 모두 다음처럼 동일한 결과를 출력한다.

> *IDLE 에디터에서 작성한 함수를 실행해 보려면 함수 정의 부분 밑에 함수를 사용하는 소스 코드까지 추가로 작성한 후 `[F5]` 를 눌러 실행하면 된다.*

```
나의 이름은 박응용입니다.
나이는 27살입니다.
남자입니다.
```

이제 초깃값이 설정된 부분을 False로 바꿔 호출해 보자.

```
say_myself("박응선", 27, False)
```

man 변수에 False 값이 대입되어 다음과 같은 결과가 출력된다.

```
나의 이름은 박응선입니다.
나이는 27살입니다.
여자입니다.
```

함수의 매개변수에 초깃값을 설정할 때 주의할 것이 하나 있다. 만약 위에서 살펴본 say_myself 함수를 다음과 같이 만들면 어떻게 될까?

```
# default2.py
def say_myself(name, man=True, age):
    print(f"나의 이름은 {name}입니다.")
    print(f"나이는 {age}살입니다.")
    if man:
        print("남자입니다.")
    else:
        print("여자입니다.")
```

이전 함수와 바뀐 부분은 초깃값을 설정한 매개변수의 위치이다. 결론을 미리 말하면 이것은 함수를 실행할 때 오류가 발생한다.

얼핏 생각하기에 위 함수를 호출하려면 다음과 같이 하면 될 것 같다.

```
say_myself("박응용", 27)
```

위와 같이 함수를 호출한다면 name 변수에는 "박응용"이 들어갈 것이다. 하지만 파이썬 인터프리터는 27을 man 매개변수와 age 매개변수 중 어느 곳에 대입해야 할지 판단하기 어려우므로 이러한 상황에서는 오류가 발생한다.

오류 메시지는 다음과 같다.

```
SyntaxError: non-default argument follows default argument
```

위 오류 메시지는 '초깃값이 없는 매개변수(age)는 초깃값이 있는 매개변수(man) 뒤에 사용할 수 없다'라는 뜻이다. 즉, 매개변수로 `(name, age, man=True)`는 되지만, `(name, man=True, age)`는 안 된다는 것이다. 초기화하고 싶은 매개변수는 항상 뒤쪽에 놓아야 한다는 것을 잊지 말자.

함수 안에서 선언한 변수의 효력 범위

함수 안에서 사용할 변수의 이름을 함수 밖에서도 동일하게 사용한다면 어떻게 될까? 이런 궁금증이 생겼던 독자라면 이번에 확실하게 답을 찾을 수 있을 것이다.

다음 예를 살펴보자.

```
# vartest.py
a = 1
def vartest(a):
    a = a +1

vartest(a)
print(a)
```

먼저 a라는 변수를 생성하고 1을 대입했다. 그리고 입력으로 들어온 값에 1을 더해 주고 결괏값은 반환하지 않는 vartest 함수를 선언했다. 그리고 vartest 함수에 입력값으로 a를 주었다. 마지막으로 a의 값을 print(a)로 출력했다. 과연 어떤 값이 출력될까?

vartest 함수에서 매개변수 a의 값에 1을 더했으므로 2가 출력될 것 같지만, 프로그램 소스를 작성해서 실행해 보면 결괏값은 1이 나온다. 그 이유는 함수 안에서 사용하는 매개변수는 함수 안에서만 사용하는 '함수만의 변수'이기 때문이다. 즉, `def vartest(a)`에서 입력값을 전달받는 매개변수 a는 함수 안에서만 사용하는 변수일 뿐, 함수 밖의 변수 a와는 전혀 상관없다는 뜻이다.

따라서 vartest 함수는 다음처럼 매개변수 이름을 hello로 바꾸어도 이전의 vartest 함수와 완전히 동일하게 동작한다.

```
def vartest(hello):
    hello = hello + 1
```

즉, 함수 안에서 사용하는 매개변수는 함수 밖의 변수 이름과는 전혀 상관없다.

다음 예를 보면 더욱 분명하게 이해할 수 있을 것이다.

```
# vartest_error.py
def vartest(a):
    a = a + 1

vartest(3)
print(a)
```

위 프로그램 소스를 에디터로 작성해서 실행하면 어떻게 될까? 오류가 발생할 것이라고 생각한 독자는 모든 것을 이해한 독자이다. vartest(3)을 수행하면 vartest 함수 안에서 a는 4가 되지만, 함수를 호출하고 난 후 print(a) 문장은 오류가 발생하게 된다. 그 이유는 print(a)에서 사용한 a 변수는 어디에도 선언되지 않았기 때문이다. 다시 한번 말하지만, 함수 안에서 선언한 매개변수는 함수 안에서만 사용될 뿐, 함수 밖에서는 사용되지 않는다. 이것을 이해하는 것은 매우 중요하다.

함수 안에서 함수 밖의 변수를 변경하는 방법

그렇다면 vartest라는 함수를 사용해서 함수 밖의 변수 a를 1만큼 증가할 수 있는 방법은 없을까? 이 질문에는 2가지 해결 방법이 있다.

1. return 사용하기

```
# vartest_return.py
a = 1
def vartest(a):
    a = a +1
    return a

a = vartest(a)
print(a)
```

첫 번째 방법은 return을 사용하는 방법이다. vartest 함수는 입력으로 들어온 값에 1을 더한 값을 반환하도록 변경했다. 따라서 `a = vartest(a)` 라고 작성하면 a에는 vartest 함수의 반환값이 대입된다.

여기에서도 물론 vartest 함수 안의 a 매개변수는 함수 밖의 a와는 다른 것이다.

2. global 명령어 사용하기

```
# vartest_global.py
a = 1
def vartest():
    global a
    a = a+1

vartest()
print(a)
```

두 번째 방법은 global 명령어를 사용하는 방법이다. 위 예에서 볼 수 있듯이 vartest 함수 안의 global a 문장은 함수 안에서 함수 밖의 a 변수를 직접 사용하겠다는 뜻이다. 하지만 프로그래밍을 할 때 global 명령어는 사용하지 않는 것이 좋다. 함수는 독립적으로 존재하는 것이 좋기 때문이다. 외부 변수에 종속적인 함수는 그다지 좋은 함수가 아니다. 따라서 되도록 global 명령어를 사용하는 이 방법은 피하고 첫 번째 방법을 사용하기를 권한다.

점프 투 파이썬

리스트나 딕셔너리는 함수에서 변경 가능하다

앞의 예제에서는 함수가 숫자나 문자열 같은 값을 받았다. 이런 값들은 함수 안에서 변경해도 함수 밖의 원래 값에는 영향을 주지 않는다. 하지만 리스트나 딕셔너리 같은 자료형은 다르다.

```
>>> def change_list(my_list):
...     my_list.append(4)  # 리스트에 값을 추가
...
>>> a = [1, 2, 3]
>>> change_list(a)
>>> print(a)
[1, 2, 3, 4]
```

위 예제를 보면 함수에서 리스트에 값을 추가했을 때 원래 리스트 a도 함께 변경되었다. 이는 리스트가 '변경 가능한(mutable)' 자료형이기 때문이다. 딕셔너리도 마찬가지로 함수 안에서 값을 변경하면 원래 딕셔너리도 함께 변경된다.

따라서 리스트나 딕셔너리를 함수에 전달할 때는 원본이 변경될 수 있다는 점을 기억하자.

lambda 예약어

lambda는 함수를 생성할 때 사용하는 예약어로, def와 동일한 역할을 한다. 보통 함수를 한 줄로 간결하게 만들 때 사용한다. 우리말로는 '람다'라고 읽고 def를 사용해야 할 정도로 복잡하지 않거나 def를 사용할 수 없는 곳에 주로 쓰인다.

사용법은 다음과 같다.

```
함수_이름 = lambda 매개변수1, 매개변수2, ... : 매개변수를_이용한_표현식
```

한번 직접 만들어 보자.

```
>>> add = lambda a, b: a+b
>>> result = add(3, 4)
>>> print(result)
7
```

> *lambda로 만든 함수는 return 명령어가 없어도 표현식의 결괏값을 반환한다.*

add는 2개의 인수를 받아 서로 더한 값을 반환하는 lambda 함수이다. 위 예제는 def를 사용한 다음 함수와 하는 일이 완전히 동일하다.

```
>>> def add(a, b):
...     return a+b
...
>>> result = add(3, 4)
>>> print(result)
7
```

함수의 독스트링(Docstring)

함수에 대한 설명을 문서화하는 방법이 독스트링이다. 함수의 첫 번째 줄에 삼중 따옴표로 둘러싼 문자열을 작성하면 된다.

```
def add(a, b):
    """
    두 숫자를 더하는 함수

    Parameters:
    a (int, float): 첫 번째 숫자
    b (int, float): 두 번째 숫자

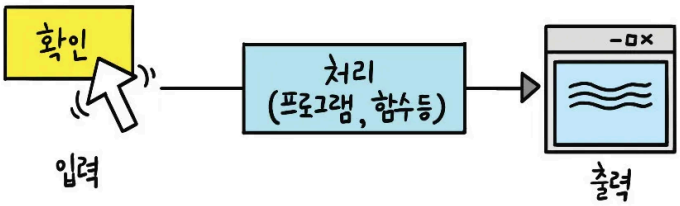
    Returns:
    int, float: 두 숫자의 합
    """
    return a + b

# 독스트링 확인하기
print(add.__doc__)
```

독스트링은 함수의 사용법을 명확하게 문서화할 때 매우 유용하다.

04-2 사용자 입출력

우리가 사용하는 대부분의 프로그램은 사용자 입력에 따라 그에 맞는 출력을 내보낸다. 예를 들어 게시판에 글을 작성한 후 [확인] 버튼을 누르면(입력) 글이 게시판에 올라간다(출력).



우리는 이미 함수 부분에서 입출력의 의미를 알아보았다. 지금부터는 좀 더 다양한 입출력 방법에 대해 알아보자.

- 사용자 입력 활용하기
  - input 사용하기
  - 프롬프트를 띄워 사용자 입력받기
- 입력값을 숫자로 변환하기
  - 정수로 변환하기
  - 실수로 변환하기
  - 한 번에 변환하기
- print 자세히 알기
  - 따옴표로 둘러싸인 문자열은 + 연산과 동일하다
  - 문자열 띄어쓰기는 쉼표로 한다
  - sep 매개변수로 구분자 설정하기
  - 한 줄에 결괏값 출력하기
- 실습: 간단한 계산기 만들기

사용자 입력 활용하기

사용자가 입력한 값을 변수에 대입하고 싶을 때는 어떻게 해야 할까?

input 사용하기

```
>>> a = input()
Life is too short, you need python
>>> a
'Life is too short, you need python'
```

input은 사용자가 키보드로 입력한 모든 것을 문자열로 저장한다.

프롬프트를 띄워 사용자 입력받기

사용자에게 입력받을 때 '숫자를 입력하세요'나 '이름을 입력하세요'처럼 안내 문구를 보여 주고 싶을 때가 있다. 그럴 때는 input()의 괄호 안에 안내 문구를 입력하면 된다.

```
input("안내_문구")
```

다음 예를 직접 입력해 보자.

```
>>> number = input("숫자를 입력하세요: ")
숫자를 입력하세요:
```

괄호 안에 입력한 문구가 프롬프트로 나타난다.

'숫자를 입력하세요'라는 프롬프트에 3을 입력하면 변수 number에 문자열 '3'이 대입된다. print(number)로 출력해서 제대로 입력되었는지 확인해 보자.

```
>>> number = input("숫자를 입력하세요: ")
숫자를 입력하세요: 3
>>> print(number)
3
```

input은 입력되는 모든 것을 문자열로 취급하므로 number는 숫자가 아닌 문자열이라는 것에 주의하자.

```
>>> type(number)
<class 'str'>
```

따라서 input으로 입력받은 값을 그대로 더하면 숫자 덧셈이 아니라 문자열 연결이 된다.

```
>>> a = input()
3
>>> b = input()
4
>>> a + b
'34'
```

> *"3" + "4" 는 숫자 7이 아니라 문자열 "34" 가 된다. 숫자 계산을 하려면 반드시 int()나 float()로 변환해야 한다.*

입력값을 숫자로 변환하기

input 함수는 항상 문자열을 반환하므로, 숫자 계산을 하려면 적절한 자료형으로 변환해야 한다.

정수로 변환하기

```
>>> age = input("나이를 입력하세요: ")
나이를 입력하세요: 25
>>> age = int(age)  # 문자열을 정수로 변환
>>> print(age + 1)
26
```

> *int() 함수는 문자열이나 실수를 정수로 변환하는 파이썬 내장 함수이다.*

실수로 변환하기

```
>>> height = input("키를 입력하세요(cm): ")
키를 입력하세요(cm): 175.5
>>> height = float(height)  # 문자열을 실수로 변환
>>> print(height / 100)  # 미터 단위로 변환
1.755
```

> *float() 함수는 문자열이나 정수를 실수로 변환하는 파이썬 내장 함수이다.*

한 번에 변환하기

input과 int(또는 float)를 한 줄에 작성할 수도 있다.

```
>>> age = int(input("나이를 입력하세요: "))
나이를 입력하세요: 25
>>> print(type(age))
<class 'int'>
```

print 자세히 알기

지금까지 사용한 print 함수의 용도는 데이터를 출력하는 것이었다. 사용 예는 다음과 같다.

```
>>> a = 123
>>> print(a)
123
>>> a = "Python"
>>> print(a)
Python
>>> a = [1, 2, 3]
>>> print(a)
[1, 2, 3]
```

이제 print 함수로 할 수 있는 일에 대해 좀 더 자세히 알아보자.

따옴표로 둘러싸인 문자열은 + 연산과 동일하다

```
>>> print("life" "is" "too short")  # 1번
lifeistoo short
>>> print("life"+"is"+"too short")  # 2번
lifeistoo short
```

위 예에서 1번과 2번은 동일한 결괏값을 출력한다. 즉, 따옴표로 둘러싸인 문자열을 연속해서 쓰면 + 연산을 한 것과 같다.

문자열 띄어쓰기는 쉼표로 한다

```
>>> print("life", "is", "too short")
life is too short
```

쉼표(,)를 사용하면 문자열 사이에 공백이 자동으로 추가된다.

sep 매개변수로 구분자 설정하기

print 함수의 sep 매개변수를 사용하면 출력할 값들 사이의 구분자를 지정할 수 있다.

```
>>> print("2025", "08", "17", sep="-")
2025-08-17
>>> print("점프", "투", "파이썬", sep=" TO ")
점프 TO 투 TO 파이썬
```

sep의 기본값은 공백(' ')이다. 앞에서 쉼표로 문자열을 구분했을 때 공백이 자동으로 추가된 것도 sep의 기본값이 공백이기 때문이다.

한 줄에 결괏값 출력하기

03-3에서 for 문을 공부할 때 만들었던 구구단 프로그램에서 보았듯이, 한 줄에 결괏값을 이어서 출력하려면 end 매개변수를 사용해 끝 문자를 지정해야 한다.

```
>>> for i in range(10):
...     print(i, end=' ')
...
0 1 2 3 4 5 6 7 8 9 >>>
```

> *마지막 출력 뒤에 줄바꿈이 없으므로 프롬프트(>>>)가 같은 줄에 나타난다. 이것은 오류가 아니니 당황하지 말자.*

end의 기본값은 줄바꿈(\n) 문자이다.

실습: 간단한 계산기 만들기

지금까지 배운 입출력 기능을 활용하여 간단한 계산기를 만들어 보자.

> *다음 예제에서 사용하는 `f"...{변수}..."` 는 02-2절에서 배운 f 문자열 포매팅이다. 문자열 안에서 변수의 값을 바로 사용할 수 있는 편리한 기능이다.*

```
# calculator.py
print("=== 간단한 계산기 ===")

# 사용자로부터 두 숫자 입력받기
num1 = float(input("첫 번째 숫자를 입력하세요: "))
num2 = float(input("두 번째 숫자를 입력하세요: "))

# 계산 결과 출력
print(f"{num1} + {num2} = {num1 + num2}")
print(f"{num1} - {num2} = {num1 - num2}")
print(f"{num1} * {num2} = {num1 * num2}")

if num2 != 0:
    print(f"{num1} / {num2} = {num1 / num2}")
else:
    print("0으로 나눌 수 없습니다.")
```

> *0으로 나누면 ZeroDivisionError 오류가 발생하므로 나눗셈 전에 `num2 != 0` 인지 확인해야 한다.*

실행 결과:

```
=== 간단한 계산기 ===
첫 번째 숫자를 입력하세요: 10
두 번째 숫자를 입력하세요: 3
10.0 + 3.0 = 13.0
10.0 - 3.0 = 7.0
10.0 * 3.0 = 30.0
10.0 / 3.0 = 3.3333333333333335
```

04-3 파일 읽고 쓰기

우리는 이 책에서 이제까지 값을 '입력'받을 때는 사용자가 직접 입력하는 방식을 사용했고 '출력'할 때는 모니터 화면에 결괏값을 출력하는 방식을 사용했다. 하지만 입출력 방법이 꼭 이것만 있는 것은 아니다. 이번에는 파일을 통한 입출력 방법에 대해 알아보자. 여기에서는 파일을 새로 만든 다음 프로그램이 만든 결괏값을 새 파일에 적어 본다. 또 파일에 적은 내용을 읽고 새로운 내용을 추가하는 방법도 알아본다.

- 파일 생성하기
- 파일을 쓰기 모드로 열어 내용 쓰기
- 파일을 읽는 여러 가지 방법
    - readline 함수 이용하기
    - readlines 함수 사용하기
    - read 함수 사용하기
    - 파일 객체를 for 문과 함께 사용하기
- 파일에 새로운 내용 추가하기
- with 문과 함께 사용하기
- with 문 안에서 만든 변수 사용하기
- 파일 처리 시 주의사항

파일 생성하기

다음 코드를 IDLE 에디터로 작성하여 실행해 보자.

```
# newfile.py
f = open("새파일.txt", 'w')
f.close()
```

프로그램을 실행한 디렉터리에 새로운 파일이 하나 생성된 것을 확인할 수 있을 것이다. 파일을 생성하기 위해 파이썬 내장 함수 open을 사용했다. open 함수는 다음과 같이 '파일 이름'과 '파일 열기 모드'를 입력값으로 받고 결괏값으로 파일 객체를 반환한다.

```
파일_객체 = open(파일_이름, 파일_열기_모드)
```

파일 열기 모드에는 다음과 같은 것들이 있다.

파일열기모드	설명
r	읽기 모드: 파일을 읽기만 할 때 사용한다.
w	쓰기 모드: 파일에 내용을 쓸 때 사용한다.
a	추가 모드: 파일의 마지막에 새로운 내용을 추가할 때 사용한다.

파일을 쓰기 모드로 열면 해당 파일이 이미 존재할 경우 원래 있던 내용이 모두 사라지고, 해당 파일이 존재하지 않으면 새로운 파일이 생성된다. 위 예에서는 디렉터리에 파일이 없는 상태에서 '새파일.txt' 파일을 쓰기 모드인 'w'로 열었기 때문에 '새파일.txt'라는 이름의 새로운 파일이 현재 디렉터리에 생성되었다.

만약 '새파일.txt' 파일을 `C:/doit` 디렉터리에 생성하고 싶다면 다음과 같이 작성해야 한다.

```
# newfile2.py
f = open("C:/doit/새파일.txt", 'w')
f.close()
```

> *맥에서는 `C:/doit/...` 대신 `~/doit/...` 처럼 홈 디렉터리 경로를 사용하면 된다.*

위 예에서 f.close()는 열려 있는 파일 객체를 닫아 주는 역할을 한다. 사실 이 문장은 생략해도 된다. 프로그램을 종료할 때 파이썬이 열려 있는 파일 객체를 자동으로 닫아 주기 때문이다. 하지만 close()를 사용해서 열려 있는 파일을 직접 닫아 주는 것이 좋다. 쓰기 모드로 열었던 파일을 닫지 않고 다시 사용하려고 하면 오류가 발생하기 때문이다.

점프 투 파이썬

파일 경로와 슬래시(/)

파이썬 코드에서 파일 경로를 표시할 때 `"C:/doit/새파일.txt"`처럼 슬래시(/)를 사용할 수 있다. 만약 역슬래시(\ )를 사용한다면 `"C:\\doit\\새파일.txt"`처럼 역슬래시를 2개 사용하거나 `r"C:\doit\새파일.txt"`와 같이 문자열 앞에 r 문자(raw string)를 덧붙여 사용해야 한다. 왜냐하면 `"C:\note\test.txt"`처럼 파일 경로에 `\n`과 같은 이스케이프 문자가 있을 경우, 줄바꿈 문자로 해석되어 의도했던 파일 경로와 달라지기 때문이다.

파일을 쓰기 모드로 열어 내용 쓰기

위 예에서는 파일을 쓰기 모드로 열기만 했을 뿐, 정작 아무것도 쓰지는 않았다. 이번에는 문자열 데이터를 파일에 직접 써 보자.

```
# write_data.py
f = open("C:/doit/새파일.txt", 'w')
for i in range(1, 11):
    data = f"{i}번째 줄입니다.\n"
    f.write(data)
f.close()
```

위 프로그램을 다음과 비교해 보자.

```
for i in range(1, 11):
    data = f"{i}번째 줄입니다.\n"
    print(data)
```

두 프로그램의 다른 점은 data를 출력하는 방법이다. 첫 번째는 모니터 화면 대신 파일에 데이터를 적는 방법, 두 번째는 모니터 화면에 데이터를 출력하는 방법이다. 두 방법의 차이점은 print 함수 대신 파일 객체 f의 write 함수를 사용한 것뿐이다.

이제 명령 프롬프트 창에서 첫 번째 예제를 실행해 보자.

```
C:\Users> cd C:\doit
C:\doit> python write_data.py
C:\doit>
```

이 프로그램을 실행한 `C:/doit` 디렉터리를 살펴보면 '새파일.txt' 파일이 생성된 것을 볼 수 있다. 파일 안에는 어떤 내용이 담겨 있는지 확인해 보자.



모니터 화면에 출력될 내용이 고스란히 파일에 들어 있는 것을 볼 수 있다.

파일을 읽는 여러 가지 방법

파이썬에는 파일을 읽는 방법이 여러 가지 있다. 하나씩 알아보자.

readline 함수 이용하기

첫 번째는 readline 함수를 사용하는 것이다. 다음 예를 살펴보자.

```
# readline_test.py
f = open("C:/doit/새파일.txt", 'r')
line = f.readline()
print(line)
f.close()
```

위는 '새파일.txt' 파일을 읽기 모드로 연 후 readline()을 사용해서 파일의 첫 번째 줄을 읽어 출력하는 예제이다. 앞에서 만든 새파일.txt 파일을 수정하거나 지우지 않았다면 위 프로그램을 실행했을 때 새파일.txt 파일의 가장 첫 번째 줄이 화면에 출력될 것이다.

```
1번째 줄입니다.

```

> *첫 번째 줄의 끝에 줄바꿈 문자 `\n` ◢이 있으므로 빈 줄이 추가로 출력된다.*

만약 모든 줄을 읽어 화면에 출력하고 싶다면 다음과 같이 작성하면 된다.

```
# readline_all.py
f = open("C:/doit/새파일.txt", 'r')
while True:
    line = f.readline()
    if not line: break
    print(line)
f.close()
```

`while True:` 무한 루프 안에서 `f.readline()`을 사용해 파일을 계속 한 줄씩 읽어 들인다. 만약 더 이상 읽을 줄이 없으면 break를 수행한다. readline()은 더 이상 읽을 줄이 없을 경우 빈 문자열('')을 반환한다.

> *한 줄씩 읽어 출력할 때 줄 끝에 `\n` 문자가 있으므로 빈 줄도 같이 출력된다.*

앞의 프로그램을 다음과 비교해 보자.

```
while True:
    data = input()
    if not data: break
    print(data)
```

위 예는 사용자의 입력을 받아 그 내용을 출력하는 경우이다. 파일을 읽어서 출력하는 예제와 비교해 보자. 입력을 받는 방식만 다르다는 것을 바로 알 수 있을 것이다. 두 번째 예는 키보드를 사용한 입력 방법, 첫 번째 예는 파일을 사용한 입력 방법이다.

readlines 함수 사용하기

두 번째 방법은 readlines 함수를 사용하는 것이다. 다음 예를 살펴보자.

```
# readlines.py
f = open("C:/doit/새파일.txt", 'r')
lines = f.readlines()
for line in lines:
    print(line)
f.close()
```

readlines 함수는 파일의 모든 줄을 읽어서 각각의 줄을 요소로 가지는 리스트를 반환한다. 따라서 위 예에서 lines는 리스트 `["1번째 줄입니다.\n", "2번째 줄입니다.\n", ... , "10번째 줄입니다.\n"]`가 된다. `f.readlines()`는 `f.readline()`과 달리 s가 하나 더 붙어 있다는 것에 유의하자.

점프 투 파이썬

줄 바꿈(`\n`) 문자 제거하기

파일을 읽을 때 줄 끝의 줄 바꿈(`\n`) 문자를 제거하고 사용해야 할 경우가 많다. 다음처럼 strip 함수를 사용하면 줄 바꿈 문자를 제거할 수 있다.

```
f = open("C:/doit/새파일.txt", 'r')
lines = f.readlines()
for line in lines:
    line = line.strip()  # 줄 끝의 줄 바꿈 문자를 제거한다.
    print(line)
f.close()
```

read 함수 사용하기

세 번째는 read 함수를 사용하는 방법이다. 다음 예를 살펴보자.

```
# read.py
f = open("C:/doit/새파일.txt", 'r')
data = f.read()
print(data)
f.close()
```

`f.read()` 는 파일의 내용 전체를 문자열로 반환한다. 따라서 위 예의 data는 파일의 전체 내용이다.

파일 객체를 for 문과 함께 사용하기

네 번째는 파일 객체를 for 문과 함께 사용하는 방법이다.

```
# read_for.py
f = open("C:/doit/새파일.txt", 'r')
for line in f:
    print(line)
f.close()
```

파일 객체(f)는 기본적으로 위와 같이 for 문과 함께 사용하여 파일을 줄 단위로 읽을 수 있다.

파일에 새로운 내용 추가하기

쓰기 모드('w')로 파일을 열 때 이미 존재하는 파일을 열면 그 파일의 내용이 모두 사라진다. 하지만 원래 있던 값을 유지하면서 새로운 값만 추가해야 할 경우도 있다. 이런 경우에는 파일을 추가 모드('a')로 열면 된다. IDLE 에디터로 다음 코드를 작성해 보

자.

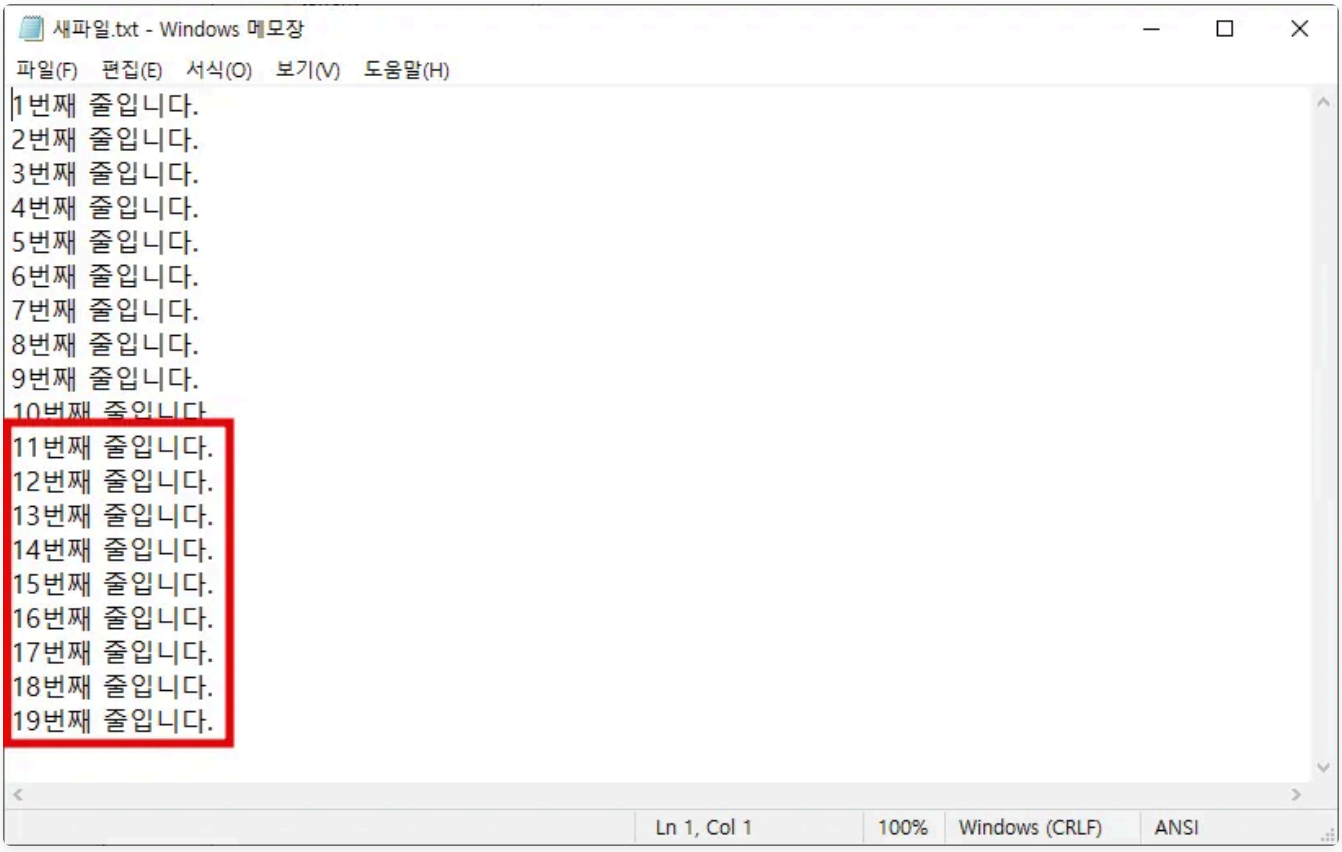
```
# add_data.py
f = open("C:/doit/새파일.txt",'a')
for i in range(11, 20):
    data = f"{i}번째 줄입니다.\n"
    f.write(data)
f.close()
```

위는 새파일.txt 파일을 추가 모드('a')로 열고 write를 사용해서 결괏값을 기존 파일에 추가해 적는 예이다. 추가 모드로 파일을 열었기 때문에 새파일.txt 파일이 원래 가지고 있던 내용 바로 다음부터 결괏값을 적기 시작한다.

다음과 같이 작성한 코드를 실행해 보자.

```
C:\Users> cd C:\doit
C:\doit> python add_data.py
C:\doit>
```

새파일.txt 파일을 열어 보면 원래 있던 내용 뒤에 새로운 내용이 추가된 것을 확인할 수 있다.



with 문과 함께 사용하기

지금까지 살펴본 예제를 보면 항상 다음과 같은 방식으로 파일을 열고 닫은 것을 알 수 있다.

```
f = open("foo.txt", 'w')
f.write("Life is too short, you need python")
f.close()
```

파일을 열면(open) 항상 닫아(close) 주어야 한다. 이렇게 파일을 열고 닫는 것을 자동으로 처리할 수 있다면 편리하지 않을까? 파이썬의 with 문이 바로 이런 역할을 해 준다. 다음 예는 with 문을 사용해서 위 예제를 다시 작성한 모습이다.

```
# file_with.py
with open("foo.txt", "w") as f:
    f.write("Life is too short, you need python")
```

with 문을 사용하면 with 블록(with 문에 속해 있는 문장)을 벗어나는 순간 열린 파일 객체 f가 자동으로 닫힌다.

점프 투 파이썬

파일이 닫혔는지 확인하기

파일 객체의 `closed` 속성을 사용하면 파일이 닫혔는지 확인할 수 있다.

```
with open("foo.txt", "w") as f:
    f.write("Hello")
    print(f.closed)  # False (아직 열려 있음)

print(f.closed)  # True (with 블록을 벗어나 자동으로 닫힘)
```

`f.closed`는 파일이 닫혔으면 True, 열려 있으면 False를 반환한다.

with 문 안에서 만든 변수 사용하기

with 문 안에서 만든 변수는 with 블록이 끝난 후에도 사용할 수 있다. 다음 예를 보자.

```
with open("test.txt", "w") as f:
    content = "Hello, Python!"
    f.write(content)

print(content)  # "Hello, Python!" 출력
```

with 블록 안에서 만든 content 변수를 블록 밖에서도 사용할 수 있다. 파이썬에서 `if`, `for`, `while`, `with` 블록은 변수의 사용 범위를 제한하지 않기 때문이다.

> *단, 파일 객체 f는 with 블록을 벗어나면 자동으로 닫히므로, 블록 밖에서 f.write()나 f.read() 같은 파일 작업은 할 수 없다. 닫힌 파일에 작업을 시도하면 오류가 발생한다.*

파일 처리 시 주의사항

한글이 포함된 파일을 다룰 때는 인코딩을 명시하는 것이 좋다.

```
# 한글 파일 쓰기
with open("한글파일.txt", "w", encoding="utf-8") as f:
    f.write("안녕하세요, 파이썬!")

# 한글 파일 읽기
with open("한글파일.txt", "r", encoding="utf-8") as f:
    content = f.read()
    print(content)
```

> *인코딩을 명시하지 않으면 운영체제마다 다른 기본 인코딩을 사용하여 한글이 깨질 수 있다.*

점프 투 파이썬

인코딩과 UTF-8

컴퓨터는 문자를 직접 저장하지 못하고 숫자로 변환해서 저장한다. 이때 어떤 문자를 어떤 숫자로 변환할지 정하는 규칙이 인코딩(Encoding)이다. 예를 들어 'A'는 65, 'B'는 66으로 저장하는 식이다.

UTF-8은 현재 가장 널리 사용되는 인코딩 방식으로, 전 세계 모든 문자(한글, 영어, 중국어, 일본어, 이모지 등)를 표현할 수 있다. 파일을 저장하거나 읽을 때 UTF-8을 사용하면 어떤 컴퓨터에서든 한글이 깨지지 않고 올바르게 표시된다.

04-4 프로그램의 입출력

명령 프롬프트를 사용해 본 독자라면 다음과 같은 명령어를 사용해 봤을 것이다.

```
C:\> type a.txt
```

type은 바로 뒤에 적힌 파일 이름을 인수로 받아 해당 파일의 내용을 출력해 주는 명령어이다. 대부분의 명령 프롬프트 명령어는 다음과 같이 인수를 전달하여 실행한다.

```
명령어 [인수1 인수2 ...]
```

> *맥/리눅스에서는 `type a.txt` 대신 `cat a.txt` 를 사용한다.*

파이썬 프로그램에도 이 기능을 적용할 수 있다.

sys 모듈 사용하기

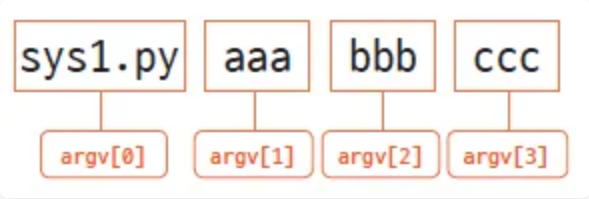
파이썬에서는 sys 모듈을 사용하여 프로그램에 인수를 전달할 수 있다. sys 모듈을 사용하려면 다음 예의 import sys처럼 import 문을 사용해야 한다.

> *모듈을 사용하고 만드는 방법은 05장에서 자세히 알아본다.*

```
# sys1.py
import sys

args = sys.argv[1:]
for i in args:
    print(i)
```

위 예제는 프로그램 실행 시 전달받은 인수를 for 문으로 차례대로 출력하는 코드이다. sys 모듈의 argv는 프로그램 실행 시 전달된 인수를 담고 있는 리스트이다. 다음과 같이 입력하면 argv[0]은 파일 이름 sys1.py가 되고, argv[1]부터는 뒤에 따라오는 인수가 차례대로 저장된다.



이 프로그램을 `C:\doit` 디렉터리에 저장한 후 인수를 전달하여 실행하면 다음과 같은 결과를 볼 수 있다.

> *명령 프롬프트를 열고 `cd C:\doit` 명령을 실행하여 `C:\doit` 디렉터리로 이동한 후 다음 명령을 실행해 보자.*

```
C:\doit>python sys1.py aaa bbb ccc
aaa
bbb
ccc
```

> *맥에서는 `python3 sys1.py aaa bbb ccc` 처럼 실행하자.*

위 예를 응용하여 다음과 같은 프로그램을 만들어 보자.

```
# sys2.py
import sys
args = sys.argv[1:]
for i in args:
    print(i.upper(), end=' ')
```

문자열 관련 함수인 upper()를 사용하여 전달된 인수를 모두 대문자로 바꾸어 주는 프로그램이다. 명령 프롬프트 창에서 다음과 같이 실행해 보자.

> *sys2.py 파일이 `C:\doit` 디렉터리 안에 있어야 한다.*

```
C:\doit>python sys2.py life is too short, you need python
```

출력 결과는 다음과 같다.

```
LIFE IS TOO SHORT, YOU NEED PYTHON
```

04장 되새김 문제

죄송합니다. 되새김 문제는 위키독스 웹 사이트에서는 비공개 처리되어 있습니다.
되새김 문제에 대한 상세한 내용은 유료 E-book 또는 종이책에서 확인하실 수 있습니다.

- 책 구입 안내